==
DJANGO NL2SQL - COMPLETE ARCHITECTURE GUIDE
Detailed Explanation of How Everything Works
==

TABLE OF CONTENTS
1. Project Overview
2. Technology Stack (All Frameworks)
3. Architecture Diagram
4. How Components Are Connected
5. Data Flow - Step by Step
6. Backend Deep Dive
7. Frontend Deep Dive
8. Database Integration
9. AI Integration
10. Communication Flow
11. File Structure Explained
12. How Each Technology Works Together
13. Request-Response Cycle
14. Security & Best Practices

==
1. PROJECT OVERVIEW
==

What is NL2SQL?
A full-stack web application that converts natural language questions into SQL queries
and executes them on a SQL Server database.

Example Workflow:
User types: "Show all customers from New York"
↓
AI converts to: "SELECT * FROM customers WHERE city = 'New York'"
↓
SQL Server executes query
↓
Results displayed in beautiful table

Project Type:
- Full-Stack Web Application
- Real-time Query Interface
- AI-Powered Database Tool
- RESTful API Architecture

==
2. TECHNOLOGY STACK (ALL FRAMEWORKS & LIBRARIES)

==

BACKEND (Python/Django)

Framework/Library	Version	Purpose
Django	5.0	Web Framework
Django REST Framework	3.14.0	API Building
django-cors-headers	4.3.1	CORS Support
pyodbc	5.0.1	SQL Server
python-dotenv	1.0.0	Environment
openai	1.12.0	AI (GPT-4)
anthropic	0.18.1	AI (Claude)

FRONTEND (JavaScript/React)

Framework/Library	Version	Purpose
React	18.2.0	UI Framework
React DOM	18.2.0	DOM Rendering
React Scripts	5.0.1	Build Tools
Axios	1.6.0	HTTP Requests

DATABASE

Microsoft SQL Server	Any	Data Storage

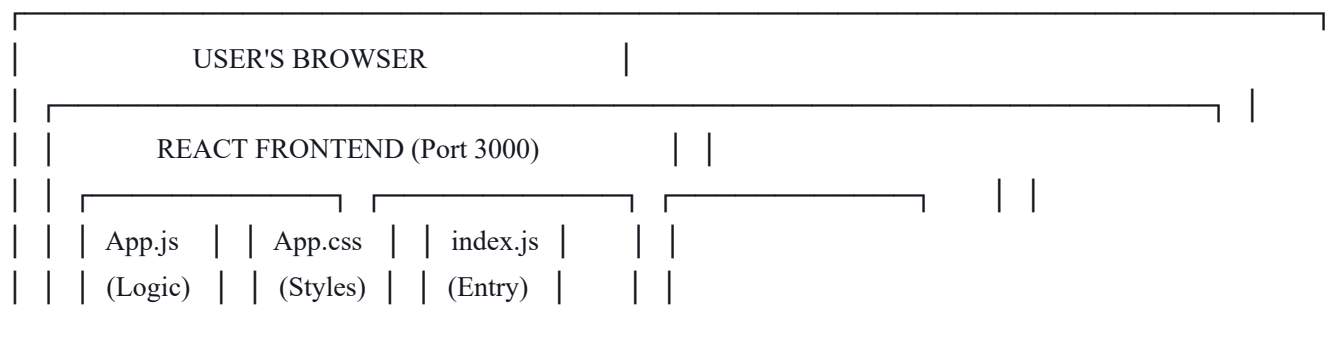
AI SERVICES

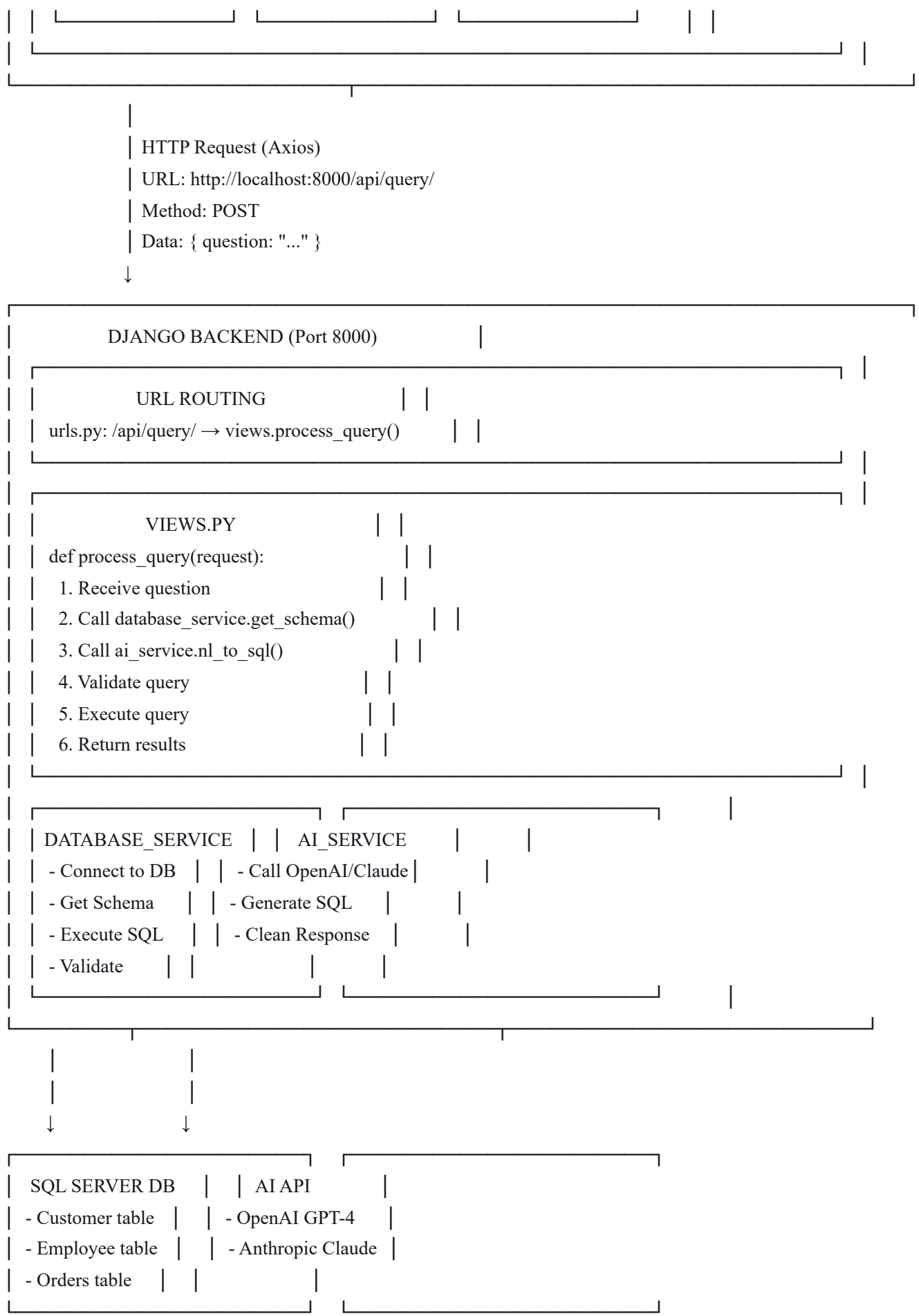
OpenAI GPT-4	Latest	NL to SQL
Anthropic Claude	Sonnet-4	NL to SQL

==

3. ARCHITECTURE DIAGRAM

==





==

4. HOW COMPONENTS ARE CONNECTED

==

Connection 1: Frontend ←→ Backend

```
┌─────────────────────────────────────────────────────────────┐
│ Technology: HTTP REST API                                   │
│ Protocol: HTTP/HTTPS                                        │
│ Format: JSON                                                │
│ Library: Axios (Frontend) ← → Django REST (Backend)         │
│                                                             │
│ Example Request:                                            │
│ axios.post('http://localhost:8000/api/query/', {            │
│   question: "Show all customers"                            │
│ })                                                          │
│                                                             │
│ Example Response:                                           │
│ {                                                           │
│   "success": true,                                          │
│   "sql": "SELECT * FROM customers",                         │
│   "data": [...]                                             │
│ }                                                           │
└─────────────────────────────────────────────────────────────┘
```

Connection 2: Backend ←→ SQL Server

```
┌─────────────────────────────────────────────────────────────┐
│ Technology: PyODBC                                          │
│ Protocol: ODBC (Open Database Connectivity)                 │
│ Driver: ODBC Driver 17 for SQL Server                       │
│                                                             │
│ Connection String:                                          │
│ DRIVER={ODBC Driver 17 for SQL Server};                     │
│ SERVER=localhost;                                           │
│ DATABASE=YourDB;                                            │
│ UID=sa;                                                     │
│ PWD=password;                                               │
│                                                             │
│ Example Code:                                               │
│ import pyodbc                                               │
│ conn = pyodbc.connect(connection_string)                    │
│ cursor = conn.cursor()                                      │
│ cursor.execute("SELECT * FROM customers")                   │
│ results = cursor.fetchall()                                 │
└─────────────────────────────────────────────────────────────┘
```

Connection 3: Backend ←→ AI APIs

```
┌─────────────────────────────────────────────────────────────┐
│ Technology: REST API Calls                                  │
│ Protocol: HTTPS                                             │
│ Format: JSON                                                │
│ Libraries: openai, anthropic                                │
```

```
│                                          │
│ OpenAI Example:                          │
│ from openai import OpenAI                │
│ client = OpenAI(api_key="sk-...")        │
│ response = client.chat.completions.create(  │
│   model="gpt-4",                         │
│   messages=[{"role": "user", "content": "..."}] │
│ )                                        │
│                                          │
│ Claude Example:                          │
│ from anthropic import Anthropic          │
│ client = Anthropic(api_key="sk-ant-...")    │
│ message = client.messages.create(        │
│   model="claude-sonnet-4-20250514",      │
│   messages=[{"role": "user", "content": "..."}] │
│ )                                        │
└──────────────────────────────────────────────────────────────────────────┘
```

==
5. DATA FLOW - STEP BY STEP
==

Complete Request-Response Cycle:

STEP 1: USER INPUT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ User types in browser:                   │
│ "Show all customers from New York"       │
│                                          │
│ Location: React Component (App.js)       │
│ State variable: question                 │
└──────────────────────────────────────────────────────────────────────────┘
                    ↓
```

STEP 2: FRONTEND SENDS REQUEST

```
┌──────────────────────────────────────────────────────────────────────────┐
│ React calls Axios:                       │
│                                          │
│ const response = await axios.post(       │
│   'http://localhost:8000/api/query/',    │
│   { question: "Show all customers from NY" }  │
│ )                                        │
│                                          │
│ Data Format: JSON                        │
│ {                                        │
│   "question": "Show all customers from NY"    │
│ }                                        │
```

```
└──────────────────────────────────────────────────────────────┘
                    ↓
```

STEP 3: DJANGO RECEIVES REQUEST

```
┌──────────────────────────────────────────────────────────────┐
│ File: nl2sql_project/urls.py                                 │
│                                                              │
│ path('api/', include('query_api.urls'))                      │
│                    ↓                                         │
│ File: query_api/urls.py                                      │
│                                                              │
│ path('query/', views.process_query)                          │
│                    ↓                                         │
│ File: query_api/views.py                                     │
│                                                              │
│ @api_view(['POST'])                                          │
│ def process_query(request):                                  │
│     question = request.data.get('question')                  │
│     # Process...                                             │
└──────────────────────────────────────────────────────────────┘
                    ↓
```

STEP 4: GET DATABASE SCHEMA

```
┌──────────────────────────────────────────────────────────────┐
│ File: query_api/database_service.py                          │
│                                                              │
│ schema = DatabaseService.get_database_schema()               │
│                                                              │
│ Executes SQL:                                                │
│ SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE                    │
│ FROM INFORMATION_SCHEMA.COLUMNS                              │
│                                                              │
│ Returns:                                                     │
│ "Database Schema:                                            │
│  Table: customers                                            │
│  Columns:                                                    │
│    - customer_id (int)                                       │
│    - name (varchar)                                          │
│    - city (varchar)"                                         │
└──────────────────────────────────────────────────────────────┘
                    ↓
```

STEP 5: CONVERT TO SQL USING AI

```
┌──────────────────────────────────────────────────────────────┐
│ File: query_api/ai_service.py                                │
│                                                              │
│ sql = AIService.natural_language_to_sql(                     │
```

```
│    question="Show all customers from NY",      │
│    database_schema=schema                      │
│ )                                              │
│                                                │
│ Sends to OpenAI/Claude:                        │
│ "Given this schema: [schema]                   │
│  Convert this question to SQL:                 │
│  Show all customers from NY"                   │
│                                                │
│ AI Returns:                                    │
│ "SELECT * FROM customers WHERE city = 'New York'" │
└────────────────────────────────────────────────┘
                    ↓
```

STEP 6: VALIDATE SQL QUERY

```
┌────────────────────────────────────────────────┐
│ File: query_api/database_service.py            │
│                                                │
│ valid = DatabaseService.validate_select_query( │
│    sql_query                                   │
│ )                                              │
│                                                │
│ Checks:                                        │
│ ✓ Query starts with SELECT                     │
│ ✓ No INSERT, UPDATE, DELETE                    │
│ ✓ No DROP, CREATE, ALTER                       │
│ ✓ No EXEC, EXECUTE                             │
│                                                │
│ Security: Prevents malicious queries           │
└────────────────────────────────────────────────┘
                    ↓
```

STEP 7: EXECUTE ON SQL SERVER

```
┌────────────────────────────────────────────────┐
│ File: query_api/database_service.py            │
│                                                │
│ results = DatabaseService.execute_query(       │
│    "SELECT * FROM customers WHERE city='NY'"   │
│ )                                              │
│                                                │
│ PyODBC executes:                               │
│ cursor.execute(sql_query)                      │
│ rows = cursor.fetchall()                       │
│                                                │
│ Converts to list of dictionaries:              │
│ [                                              │
│   {"id": 1, "name": "John", "city": "NY"},     │
```

```
│   {"id": 2, "name": "Jane", "city": "NY"}        │
│ ]                                                 │
└──────────────────────────────────────────────────┘
                    ↓
```

STEP 8: RETURN RESPONSE TO FRONTEND

```
┌──────────────────────────────────────────────────┐
│ File: query_api/views.py                         │
│                                                  │
│ return Response({                                │
│     "success": true,                             │
│     "question": "Show all customers from NY",    │
│     "sql": "SELECT * FROM customers...",         │
│     "data": [...],                               │
│     "rowCount": 2                                │
│ })                                               │
│                                                  │
│ Format: JSON over HTTP                           │
└──────────────────────────────────────────────────┘
                    ↓
```

STEP 9: FRONTEND DISPLAYS RESULTS

```
┌──────────────────────────────────────────────────┐
│ File: frontend/src/App.js                        │
│                                                  │
│ setResult(response.data)                         │
│                                                  │
│ React renders:                                   │
│ - SQL query in code block                        │
│ - Data in table format                           │
│ - Row count                                      │
│ - Export button                                  │
└──────────────────────────────────────────────────┘
```

==========
6. BACKEND DEEP DIVE (Django)
==========

What is Django?
Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design.

Why Django?
✓ Built-in admin interface
✓ ORM (Object-Relational Mapping)
✓ Security features (CSRF, SQL injection protection)
✓ Scalable architecture

✓ Large community and packages

Django Components in Our Project:

1. DJANGO PROJECT (nl2sql_project/)

```
┌─────────────────────────────────────────────────────────────┐
│ settings.py  → Configuration                                │
│ - Database settings                                         │
│ - Installed apps                                            │
│ - Middleware                                                │
│ - CORS configuration                                        │
│ - AI provider settings                                      │
│                                                             │
│ urls.py      → URL routing                                  │
│ - Maps URLs to views                                        │
│ - /api/ → query_api app                                     │
│                                                             │
│ wsgi.py      → Production server interface                  │
│ asgi.py      → Async server interface                       │
└─────────────────────────────────────────────────────────────┘
```

2. DJANGO APP (query_api/)

```
┌─────────────────────────────────────────────────────────────┐
│ views.py             → Request handlers                     │
│ - process_query()    → Main NL2SQL endpoint                 │
│ - health_check()     → API health                           │
│ - get_schema()       → Database schema                      │
│ - test_connection()  → DB connection test                   │
│ - test_sql()         → Direct SQL execution                 │
│                                                             │
│ urls.py              → App URL routing                      │
│ - /api/query/        → process_query                        │
│ - /api/health/       → health_check                         │
│                                                             │
│ database_service.py  → Database operations                  │
│ - Class-based service layer                                 │
│ - Connection management                                     │
│ - Schema retrieval                                          │
│ - Query execution                                           │
│ - Query validation                                          │
│                                                             │
│ ai_service.py        → AI operations                        │
│ - OpenAI integration                                        │
│ - Claude integration                                        │
│ - Prompt engineering                                        │
│ - Response cleaning                                         │
│                                                             │
```

```
│ models.py          → Data models (not used)  │
│ admin.py           → Admin interface         │
│ apps.py            → App configuration       │
└──────────────────────────────────────────────┘
```

Django REST Framework (DRF)

What it does:
- Converts Python functions to REST API endpoints
- Handles JSON serialization/deserialization
- Provides @api_view decorator
- Manages HTTP methods (GET, POST, etc.)

Example:
```python
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(['POST'])
def process_query(request):
    data = request.data  # Automatically parses JSON
    # Process...
    return Response({     # Automatically converts to JSON
        "success": True,
        "data": results
    })
```

PyODBC

What it does:
- Python database API
- Connects to SQL Server using ODBC
- Executes SQL queries
- Fetches results

Connection Process:
```python
1. Build connection string
   "DRIVER={ODBC Driver 17};SERVER=localhost;..."

2. Create connection
   conn = pyodbc.connect(connection_string)

3. Create cursor
   cursor = conn.cursor()
```

4. Execute query
   cursor.execute("SELECT * FROM table")

5. Fetch results
   rows = cursor.fetchall()

6. Close connection
   cursor.close()
   conn.close()
```

## ========================================
## 7. FRONTEND DEEP DIVE (React)
## ========================================

### What is React?
React is a JavaScript library for building user interfaces, developed by Facebook.
It uses a component-based architecture.

### Why React?
✓ Component reusability
✓ Virtual DOM (fast rendering)
✓ Large ecosystem
✓ Easy state management
✓ Popular and well-supported

### React Components in Our Project:

1. ENTRY POINT (index.js)

```
┌──────────────────────────────────────────────────────────────┐
│ import React from 'react'                                    │
│ import ReactDOM from 'react-dom/client'                      │
│ import App from './App'                                      │
│                                                              │
│ const root = ReactDOM.createRoot(                            │
│   document.getElementById('root')                            │
│ )                                                            │
│                                                              │
│ root.render(<App />)                                         │
│                                                              │
│ What it does:                                                │
│ - Finds  in HTML                              │
│ - Renders App component inside it                            │
└──────────────────────────────────────────────────────────────┘
```

2. MAIN COMPONENT (App.js)

```
┌──────────────────────────────────────────────────────────────┐
```
```

```
┌──────────────────────────────────────────────────────────────────────┐
│ State Variables (useState):                                          │
│ - question       → User's input                                      │
│ - loading        → Is request in progress?                           │
│ - result         → API response data                                 │
│ - error          → Error messages                                    │
│ - showSQL        → Toggle SQL display                                │
│                                                                      │
│ Functions:                                                           │
│ - handleSubmit()       → Form submission                             │
│ - handleExampleClick() → Click example question                      │
│ - renderTable()        → Display results                             │
│ - exportToCSV()        → Download CSV                                │
│                                                                      │
│ JSX (HTML-like syntax):                                              │
│ return (                                                             │
│   <div className="App">                                              │
│     <header>...</header>                                             │
│     <form onSubmit={handleSubmit}>...</form>                         │
│     {result && <Table data={result.data} />}                         │
│   </div>                                                             │
│ )                                                                    │
└──────────────────────────────────────────────────────────────────────┘
```

React Concepts Used:

1. STATE MANAGEMENT
```javascript
const [question, setQuestion] = useState('');

// Reading state
console.log(question);

// Updating state
setQuestion('New value');

// Why? React re-renders component when state changes
```

2. EVENT HANDLING
```javascript
<button onClick={handleClick}>Click Me</button>

function handleClick() {
  // Handle click
}
```

3. CONDITIONAL RENDERING
```javascript
{error && <div className="error">{error}</div>}
// Only shows if error exists

{loading ? <Spinner /> : <Button />}
// Shows spinner while loading, button otherwise
```

4. MAPPING (Lists)
```javascript
{data.map((row, index) => (
  <tr key={index}>
    <td>{row.name}</td>
  </tr>
))}
// Creates table row for each data item
```

Axios (HTTP Client)

What it does:
- Makes HTTP requests from browser
- Handles JSON automatically
- Promise-based (async/await)
- Automatic error handling

Example:
```javascript
// POST request
const response = await axios.post(
  'http://localhost:8000/api/query/',
  { question: "..." },
  { headers: { 'Content-Type': 'application/json' } }
);

// Response
console.log(response.data);
// { success: true, sql: "...", data: [...] }
```

==
8. DATABASE INTEGRATION (SQL Server)
==

How We Connect to SQL Server:

1. ODBC DRIVER

```
┌──────────────────────────────────────────────────────────────────────┐
│ What: Open Database Connectivity                                     │
│ Why: Industry standard for database connections                      │
│ Driver: ODBC Driver 17 for SQL Server                                │
│                                                                      │
│ Installation:                                                        │
│ Windows: Download from Microsoft                                     │
│ Linux: sudo apt-get install msodbcsql17                              │
└──────────────────────────────────────────────────────────────────────┘
```

2. CONNECTION STRING

```
┌──────────────────────────────────────────────────────────────────────┐
│ Format:                                                              │
│ DRIVER={ODBC Driver 17 for SQL Server};                              │
│ SERVER=localhost;                                                    │
│ DATABASE=CustomerDB;                                                 │
│ UID=sa;                                                              │
│ PWD=password123;                                                     │
│ TrustServerCertificate=yes;                                          │
│ Encrypt=yes;                                                         │
│                                                                      │
│ Components:                                                          │
│ - DRIVER: Which ODBC driver to use                                   │
│ - SERVER: Database server address                                    │
│ - DATABASE: Database name                                            │
│ - UID: Username                                                      │
│ - PWD: Password                                                      │
│ - TrustServerCertificate: Skip cert validation                       │
│ - Encrypt: Use encryption                                            │
└──────────────────────────────────────────────────────────────────────┘
```

3. SCHEMA DETECTION

```
┌──────────────────────────────────────────────────────────────────────┐
│ Query Used:                                                          │
│ SELECT                                                               │
│   t.TABLE_NAME,                                                      │
│   c.COLUMN_NAME,                                                     │
│   c.DATA_TYPE                                                        │
│ FROM INFORMATION_SCHEMA.TABLES t                                     │
│ INNER JOIN INFORMATION_SCHEMA.COLUMNS c                              │
│   ON t.TABLE_NAME = c.TABLE_NAME                                     │
│ WHERE t.TABLE_TYPE = 'BASE TABLE'                                    │
│ ORDER BY t.TABLE_NAME, c.ORDINAL_POSITION                            │
│                                                                      │
│ What it returns:                                                     │
│ Table: customers                                                     │
```

```
│ - customer_id (int)                    │
│ - name (varchar)                       │
│ - email (varchar)                      │
│                                        │
│ Table: orders                          │
│ - order_id (int)                       │
│ - customer_id (int)                    │
│ - total (decimal)                      │
└────────────────────────────────────────┘
```

==
9. AI INTEGRATION (OpenAI / Claude)
==

Why Use AI?
To convert natural language questions to SQL queries

How It Works:

1. PROMPT ENGINEERING

```
┌──────────────────────────────────────────────────────────────┐
│ We send to AI:                                               │
│                                                              │
│ "You are a SQL expert. Convert the following                 │
│  natural language question to SQL Server query.              │
│                                                              │
│  Database Schema:                                            │
│  Table: customers                                            │
│    - customer_id (int)                                       │
│    - name (varchar)                                          │
│    - city (varchar)                                          │
│                                                              │
│  Question: Show all customers from New York                  │
│                                                              │
│  Important:                                                  │
│  1. Return ONLY the SQL query                                │
│  2. Use TOP instead of LIMIT for SQL Server                  │
│  3. Ensure query is SELECT only                              │
│                                                              │
│  SQL Query:"                                                 │
│                                                              │
│ AI Returns:                                                  │
│ "SELECT * FROM customers WHERE city = 'New York'"            │
└──────────────────────────────────────────────────────────────┘
```

2. OPENAI INTEGRATION

```python
```

```
from openai import OpenAI

client = OpenAI(api_key="sk-...")

response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {
            "role": "system",
            "content": "You are a SQL expert..."
        },
        {
            "role": "user",
            "content": prompt
        }
    ],
    temperature=0,      # More deterministic
    max_tokens=500      # Response length limit
)

sql_query = response.choices[0].message.content
```

3. CLAUDE INTEGRATION
```python
from anthropic import Anthropic

client = Anthropic(api_key="sk-ant-...")

message = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1000,
    temperature=0,
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)

sql_query = message.content[0].text
```

Why Two AI Providers?
- Flexibility: Switch based on preference
- Reliability: Fallback if one is down

- Cost: Different pricing models
- Features: Different strengths

==
10. COMMUNICATION FLOW
==

HTTP Communication (Frontend ↔ Backend)

1. REQUEST FORMAT

```
┌──────────────────────────────────────────────────────────────────────┐
│ HTTP Method: POST                                                    │
│ URL: http://localhost:8000/api/query/                                │
│ Headers:                                                             │
│   Content-Type: application/json                                     │
│ Body:                                                                │
│   {                                                                  │
│     "question": "Show all customers"                                 │
│   }                                                                  │
└──────────────────────────────────────────────────────────────────────┘
```

2. RESPONSE FORMAT

```
┌──────────────────────────────────────────────────────────────────────┐
│ HTTP Status: 200 OK                                                  │
│ Headers:                                                             │
│   Content-Type: application/json                                     │
│ Body:                                                                │
│   {                                                                  │
│     "success": true,                                                 │
│     "question": "Show all customers",                                │
│     "sql": "SELECT * FROM customers",                                │
│     "data": [                                                        │
│       {"id": 1, "name": "John"},                                     │
│       {"id": 2, "name": "Jane"}                                      │
│     ],                                                               │
│     "rowCount": 2                                                    │
│   }                                                                  │
└──────────────────────────────────────────────────────────────────────┘
```

CORS (Cross-Origin Resource Sharing)

Problem:
Browser security blocks requests from localhost:3000 to localhost:8000
(different ports = different origins)

Solution:
Django CORS Headers middleware

Configuration in settings.py:
```python
INSTALLED_APPS = [
    'corsheaders',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
]

CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
]
```

What it does:
- Adds CORS headers to responses
- Allows frontend to make requests
- Prevents unauthorized access

==
11. FILE STRUCTURE EXPLAINED
==

BACKEND STRUCTURE
```
backend/
│
├── manage.py                  # Django CLI tool
│   └── Purpose: Run server, migrations, commands
│
├── nl2sql_project/            # Django Project (Settings)
│   ├── __init__.py            # Python package marker
│   ├── settings.py            # MAIN CONFIGURATION
│   │   ├── Database config
│   │   ├── Installed apps
│   │   ├── Middleware
│   │   ├── CORS settings
│   │   └── AI provider settings
│   │
│   ├── urls.py                # Main URL routing
│   │   └── Routes /api/ to query_api app
│   │
│   ├── wsgi.py                # Production server
│   └── asgi.py                # Async server
│
```

```
├── query_api/                # Django App (Business Logic)
│   ├── __init__.py
│   │
│   ├── views.py              # API ENDPOINTS (Request Handlers)
│   │   ├── health_check()      → GET /api/health/
│   │   ├── test_connection()   → GET /api/test-connection/
│   │   ├── get_schema()        → GET /api/schema/
│   │   ├── process_query()     → POST /api/query/
│   │   └── test_sql()          → POST /api/test-sql/
│   │
│   ├── urls.py               # App URL routing
│   │   └── Maps endpoints to views
│   │
│   ├── database_service.py     # DATABASE LAYER
│   │   ├── get_connection_string()
│   │   ├── get_database_schema()
│   │   ├── execute_query()
│   │   ├── validate_select_query()
│   │   └── test_connection()
│   │
│   ├── ai_service.py           # AI LAYER
│   │   ├── natural_language_to_sql()
│   │   ├── _openai_to_sql()
│   │   └── _anthropic_to_sql()
│   │
│   ├── models.py             # Data models (not used)
│   ├── admin.py              # Admin interface
│   └── apps.py               # App configuration
│
├── venv/                     # Virtual environment
├── requirements.txt          # Python dependencies
└── .env                      # Environment variables
```

FRONTEND STRUCTURE

```
frontend/
│
├── public/
│   └── index.html            # HTML Template
│       └── Contains <div id="root">
│
├── src/
│   ├── index.js              # ENTRY POINT
│   │   ├── Imports React
│   │   ├── Imports App component
│   │   └── Renders App into root div
```

```
│   │
│   ├── index.css              # GLOBAL STYLES
│   │   ├── Body styling
│   │   ├── Background gradient
│   │   └── Font settings
│   │
│   ├── App.js                 # MAIN COMPONENT
│   │   ├── State management
│   │   ├── Event handlers
│   │   ├── API calls
│   │   └── UI rendering
│   │
│   └── App.css                # COMPONENT STYLES
│       ├── Layout styles
│       ├── Component styles
│       ├── Animations
│       └── Responsive design
│
├── package.json               # npm dependencies
├── package-lock.json          # Locked versions
├── .env                       # Environment variables
└── node_modules/              # Installed packages
```

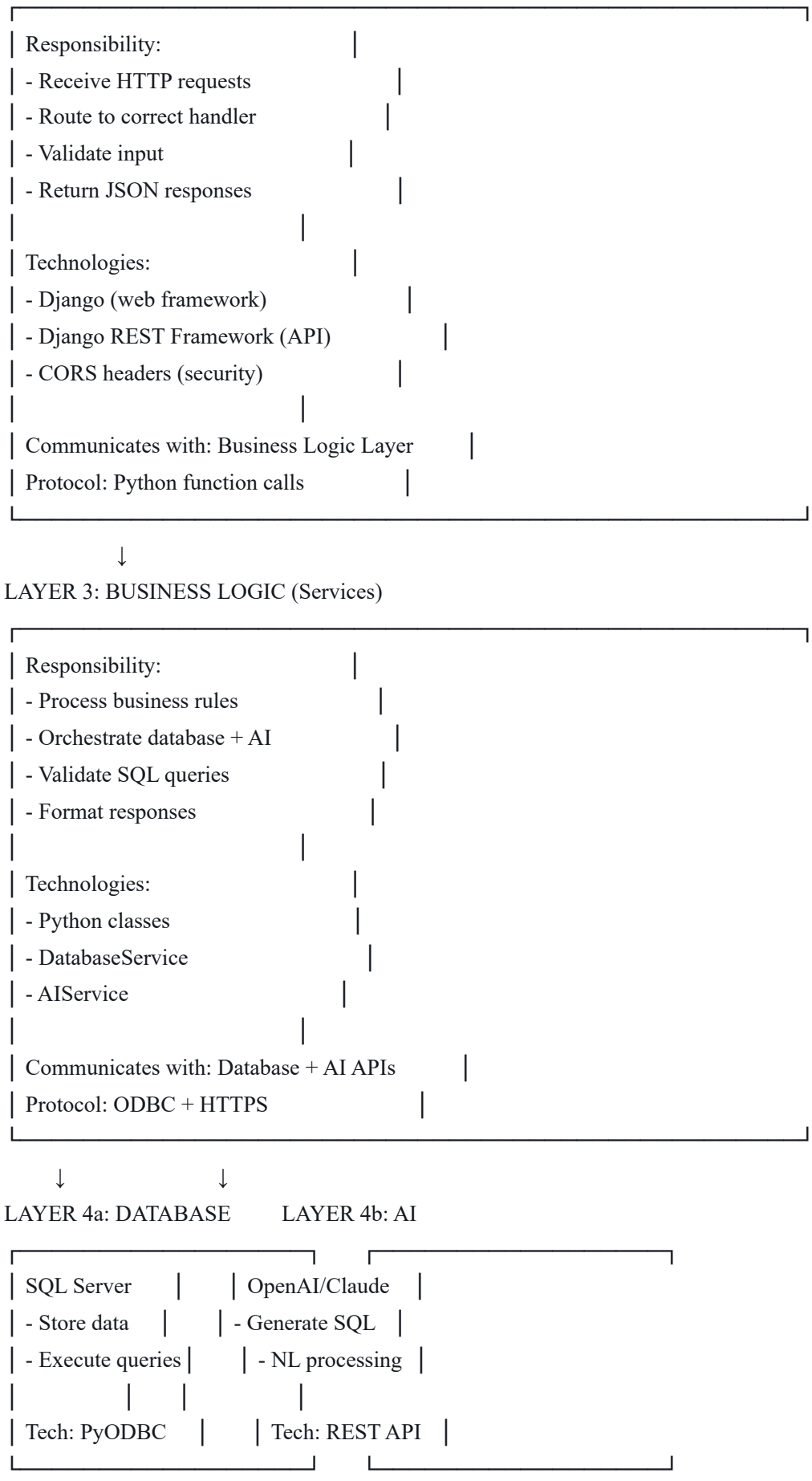
==
12. HOW EACH TECHNOLOGY WORKS TOGETHER
==

Layer-by-Layer Interaction:

LAYER 1: PRESENTATION (React)

```
┌─────────────────────────────────────────────────────────────┐
│ Responsibility:                                             │
│ - Display UI                                                │
│ - Handle user input                                         │
│ - Show results                                              │
│                                                             │
│ Technologies:                                               │
│ - React (UI framework)                                      │
│ - CSS (styling)                                             │
│ - Axios (HTTP client)                                       │
│                                                             │
│ Communicates with: API Layer (Django)                       │
│ Protocol: HTTP/JSON                                         │
└─────────────────────────────────────────────────────────────┘
                          ↓
```

LAYER 2: API (Django REST Framework)



Technology Integration Example:

USER TYPES QUESTION

↓

REACT (State Update)

useState, onChange event
↓
AXIOS (HTTP Request)
POST to Django API
↓
DJANGO (Routing)
urls.py → views.py
↓
DRF (Request Parsing)
request.data extraction
↓
PYTHON (Business Logic)
views.process_query()
↓
DATABASE SERVICE (Schema)
PyODBC → SQL Server
↓
AI SERVICE (NL to SQL)
OpenAI/Claude API
↓
DATABASE SERVICE (Execute)
PyODBC → SQL Server
↓
DRF (JSON Response)
Return Response({...})
↓
AXIOS (Promise Resolution)
.then(response => ...)
↓
REACT (State Update)
setResult(response.data)
↓
REACT (Re-render)
Display table

==
13. REQUEST-RESPONSE CYCLE (DETAILED)
==

Complete Lifecycle of One Request:

TIME: 0ms - USER ACTION

```
┌──────────────────────────────────────────────────────────────────────┐
│ Event: User clicks "Generate Query" button                           │
│ Location: Browser                                                    │
│ Component: App.js                                                    │
│ Function: handleSubmit()                                             │
```

```
└──────────────────────────────────────────────────────────────┘
```

TIME: 10ms - FRONTEND PROCESSING

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Validate input                                       │
│ Code:                                                        │
│   if (!question.trim()) {                                    │
│     setError('Please enter a question')                      │
│     return                                                   │
│   }                                                          │
│   setLoading(true)                                           │
└──────────────────────────────────────────────────────────────┘
```

TIME: 20ms - HTTP REQUEST

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Axios sends POST request                             │
│ Code:                                                        │
│   const response = await axios.post(                         │
│     'http://localhost:8000/api/query/',                      │
│     { question: question }                                   │
│   )                                                          │
│                                                              │
│ Network: Browser → Django (localhost)                        │
│ Data: JSON payload                                           │
└──────────────────────────────────────────────────────────────┘
```

TIME: 30ms - DJANGO RECEIVES

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Django receives HTTP request                         │
│ Flow:                                                        │
│   1. WSGI server receives request                            │
│   2. Middleware processes (CORS, etc)                       │
│   3. URL router matches /api/query/                          │
│   4. Routes to query_api.views.process_query                 │
└──────────────────────────────────────────────────────────────┘
```

TIME: 40ms - VIEW PROCESSING

```
┌──────────────────────────────────────────────────────────────┐
│ Action: process_query() executes                             │
│ Code:                                                        │
│   question = request.data.get('question')                    │
│   # Extract from JSON body                                   │
└──────────────────────────────────────────────────────────────┘
```

TIME: 50ms - GET DATABASE SCHEMA

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Call DatabaseService                                 │
```

```
┌──────────────────────────────────────────────────────────────┐
│ Code:                                                        │
│   schema = DatabaseService.get_database_schema()             │
│                                                              │
│ Internal:                                                    │
│   1. Connect to SQL Server (PyODBC)                          │
│   2. Execute INFORMATION_SCHEMA query                        │
│   3. Format results as text                                  │
│   4. Return schema string                                    │
│                                                              │
│ Time: ~100ms (database round-trip)                           │
└──────────────────────────────────────────────────────────────┘
```

TIME: 150ms - CALL AI API

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Convert to SQL using AI                              │
│ Code:                                                        │
│   sql = AIService.natural_language_to_sql(                   │
│     question, schema                                         │
│   )                                                          │
│                                                              │
│ Internal:                                                    │
│   1. Build prompt with schema + question                     │
│   2. Call OpenAI/Claude API (HTTPS)                          │
│   3. Wait for response                                       │
│   4. Clean/parse SQL from response                           │
│   5. Return SQL string                                       │
│                                                              │
│ Time: ~1-2 seconds (AI processing)                           │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2000ms - VALIDATE SQL

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Security check                                       │
│ Code:                                                        │
│   if not DatabaseService.validate_select_query(              │
│     sql_query                                                │
│   ):                                                         │
│     return error                                             │
│                                                              │
│ Checks: Must start with SELECT, no dangerous                 │
│         keywords                                             │
│                                                              │
│ Time: ~1ms                                                   │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2001ms - EXECUTE SQL

```
┌──────────────────────────────────────────────────────────────┐
```

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Run query on database                                │
│ Code:                                                        │
│   results = DatabaseService.execute_query(                   │
│     sql_query                                                │
│   )                                                          │
│                                                              │
│ Internal:                                                    │
│   1. Connect to SQL Server                                   │
│   2. Execute SQL query                                       │
│   3. Fetch all rows                                          │
│   4. Convert to list of dicts                                │
│   5. Close connection                                        │
│                                                              │
│ Time: ~50-500ms (depends on query)                           │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2500ms - BUILD RESPONSE

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Create JSON response                                 │
│ Code:                                                        │
│   return Response({                                          │
│     "success": True,                                         │
│     "question": question,                                    │
│     "sql": sql_query,                                        │
│     "data": results,                                         │
│     "rowCount": len(results)                                 │
│   })                                                         │
│                                                              │
│ DRF serializes to JSON automatically                         │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2510ms - HTTP RESPONSE

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Send response back to browser                        │
│ Network: Django → Browser                                    │
│ Data: JSON over HTTP                                         │
│ Size: ~1-100KB (depends on results)                          │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2520ms - FRONTEND RECEIVES

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Axios promise resolves                               │
│ Code:                                                        │
│   const response = await axios.post(...)                     │
│   // response contains data                                  │
│   setResult(response.data)                                   │
│   setLoading(false)                                          │
```

```
└──────────────────────────────────────────────────────────────┘
```

TIME: 2530ms - REACT RE-RENDERS

```
┌──────────────────────────────────────────────────────────────┐
│ Action: State change triggers re-render                      │
│ Code:                                                        │
│   {result && (                                               │
│     <div>                                                    │
│       <SQLDisplay sql={result.sql} />                        │
│       <Table data={result.data} />                           │
│     </div>                                                   │
│   )}                                                         │
│                                                              │
│ React updates DOM efficiently (Virtual DOM)                  │
└──────────────────────────────────────────────────────────────┘
```

TIME: 2540ms - USER SEES RESULTS

```
┌──────────────────────────────────────────────────────────────┐
│ Action: Browser displays results                             │
│ Elements:                                                    │
│ - SQL query in code block                                    │
│ - Data table with rows                                       │
│ - Export button                                              │
│ - Row count                                                  │
└──────────────────────────────────────────────────────────────┘
```

TOTAL TIME: ~2.5 seconds

==========

14. SECURITY & BEST PRACTICES

==========

Security Measures Implemented:

1. SQL INJECTION PREVENTION

```
┌──────────────────────────────────────────────────────────────┐
│ Method: Query Validation                                     │
│                                                              │
│ Check 1: Only allow SELECT                                   │
│   if not sql.upper().startswith('SELECT'):                   │
│     reject                                                   │
│                                                              │
│ Check 2: Block dangerous keywords                            │
│   forbidden = ['INSERT', 'UPDATE', 'DELETE',                 │
│                'DROP', 'CREATE', 'ALTER']                    │
│   if any(keyword in sql for keyword in forbidden):           │
│     reject                                                   │
```

```
│                                 │
│ Why: Prevents malicious SQL execution │
└─────────────────────────────────────────┘
```

2. ENVIRONMENT VARIABLES

```
┌─────────────────────────────────────────┐
│ Method: .env files (not in Git)         │
│                                 │
│ Secrets stored:                 │
│ - Database passwords            │
│ - API keys                      │
│ - Secret keys                   │
│                                 │
│ Why: Never commit sensitive data to Git │
└─────────────────────────────────────────┘
```

3. CORS PROTECTION

```
┌─────────────────────────────────────────┐
│ Method: django-cors-headers             │
│                                 │
│ Configuration:                  │
│   CORS_ALLOWED_ORIGINS = [              │
│     "http://localhost:3000"             │
│   ]                             │
│                                 │
│ Why: Only allow requests from trusted origins │
└─────────────────────────────────────────┘
```

4. CSRF PROTECTION

```
┌─────────────────────────────────────────┐
│ Method: Django middleware               │
│                                 │
│ How: Django validates CSRF tokens       │
│                                 │
│ Why: Prevent cross-site request forgery │
└─────────────────────────────────────────┘
```

5. INPUT VALIDATION

```
┌─────────────────────────────────────────┐
│ Method: Check before processing         │
│                                 │
│ Checks:                         │
│ - Question not empty            │
│ - Valid JSON format             │
│ - Reasonable length             │
│                                 │
│ Why: Prevent malformed requests         │
```

Best Practices Used:

1. SEPARATION OF CONCERNS

```
┌──────────────────────────────────────────────────────────┐
│ Views:         Handle HTTP requests/responses            │
│ Services:      Business logic                            │
│ Models:        Data structure (not used here)            │
│                                                          │
│ Why: Maintainable, testable code                         │
└──────────────────────────────────────────────────────────┘
```

2. DRY (Don't Repeat Yourself)

```
┌──────────────────────────────────────────────────────────┐
│ Example:                                                 │
│ - Database connection logic in ONE place                 │
│ - AI calling logic in ONE place                          │
│ - Reused across multiple endpoints                       │
│                                                          │
│ Why: Easier to maintain and update                       │
└──────────────────────────────────────────────────────────┘
```

3. ERROR HANDLING

```
┌──────────────────────────────────────────────────────────┐
│ Method: try/except blocks everywhere                     │
│                                                          │
│ try:                                                     │
│   # Risky operation                                      │
│ except Exception as e:                                   │
│   return Response({                                      │
│     "success": False,                                    │
│     "error": str(e)                                      │
│   })                                                     │
│                                                          │
│ Why: Graceful degradation, user-friendly errors          │
└──────────────────────────────────────────────────────────┘
```

4. ENVIRONMENT-BASED CONFIGURATION

```
┌──────────────────────────────────────────────────────────┐
│ Development:  DEBUG=True, localhost                      │
│ Production:   DEBUG=False, real domain                   │
│                                                          │
│ Why: Different settings for different                    │
│      environments                                        │
└──────────────────────────────────────────────────────────┘
```

==

SUMMARY - HOW IT ALL WORKS TOGETHER

==

1. User types question in React frontend
2. React calls Django API via Axios (HTTP/JSON)
3. Django routes request to views.py
4. Views calls DatabaseService to get schema
5. Views calls AIService to convert question to SQL
6. AI API (OpenAI/Claude) generates SQL query
7. Views validates SQL (security check)
8. DatabaseService executes SQL on SQL Server
9. Results returned as Python list of dicts
10. Django converts to JSON and sends response
11. React receives data via Axios
12. React updates state and re-renders
13. User sees results in beautiful table

Every piece connects through well-defined interfaces:
- Frontend ↔ Backend: HTTP REST API (JSON)
- Backend ↔ Database: PyODBC (ODBC protocol)
- Backend ↔ AI: REST API (HTTPS/JSON)

This creates a scalable, maintainable, secure application!

==

END OF DOCUMENTATION

==